Unified Backend

Proposal — replacing iboplayer, bobplayer and cr-unified with one clean, multi-tenant backend

1. Summary

Today three backends — **iboplayer-backend**, **bobplayer-backend** and **cr-unified-backend** (~100k lines in total) — serve our apps. They were copied from the same original and have drifted apart: the same logic exists several times, data is stored inconsistently, and there are serious security and reliability problems. Every fix must be made up to three times.

Proposal: build **one** backend that serves every app (tenant) from a single codebase, with a **central database** for platform-wide data and **one database per app**, then migrate all existing data into it.

> **The constraint that shapes everything:** many installed apps (Samsung/LG TVs, Android, iOS, Roku, set-top boxes) can't be updated. They must keep calling the same URLs, with the same encoded payloads, and receive identical responses. The external store must keep working unchanged too.

Approach — two phases:

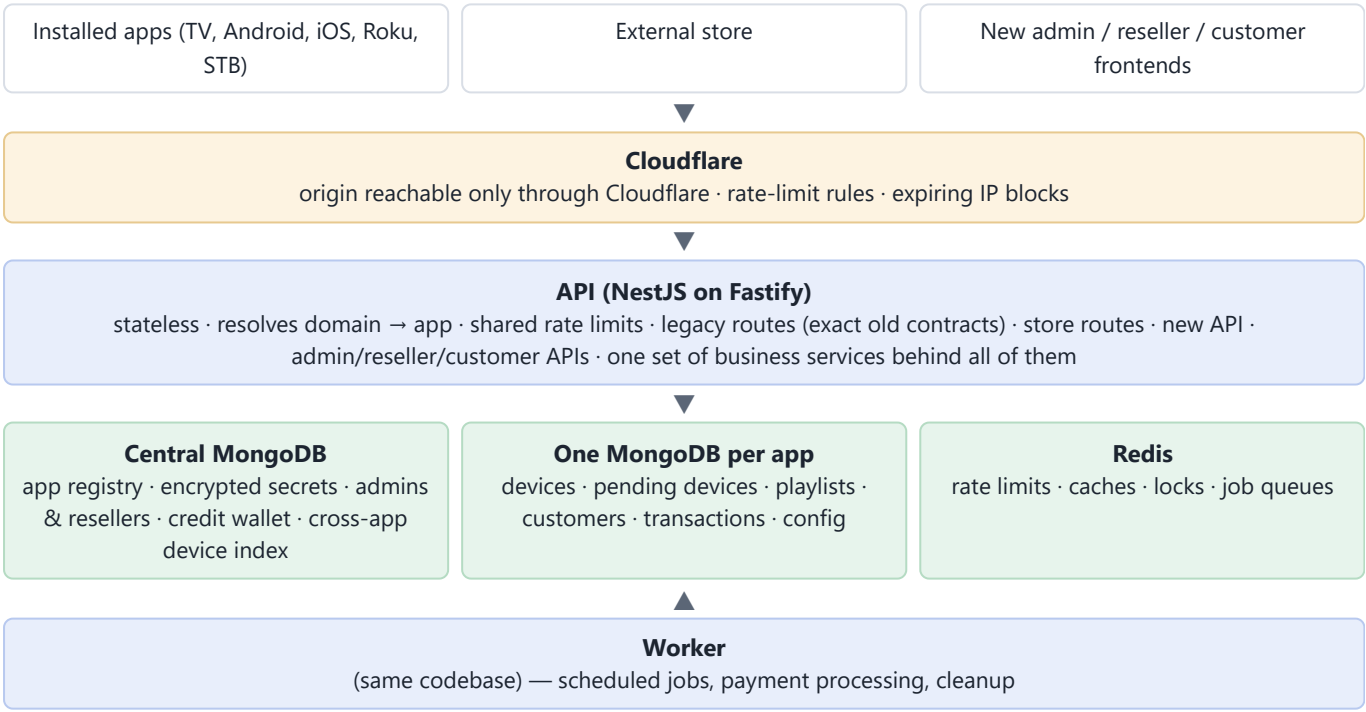
- **Phase 1 — faithful clean rebuild:** new architecture and clean code, while every contract the apps and the store see stays exactly the same. Known bugs that must be copied for compatibility are tagged in the code.
- **Phase 2 — fixes and cleanup (required before go-live):** fix every tagged bug and security gap, and remove dead endpoints after checking them against the real apps.

2. Problems in the old backends and how the new one solves them

Area	Problem today	Solution in the new backend
Code	Controller files of 4,000–9,000 lines mixing HTTP, decryption, auth, database and business rules; the same logic written many times (e.g. ~8 versions of playlist create/update/delete); a shared module copied by hand into three repos.	Small modules with one responsibility each; business rules written once in services and covered by tests; one repository.
Apps in code	App-specific behaviour hardcoded as `if (tenant === '…')` branches; adding an app means changing code.	Apps, their domains, secrets and settings are records in a central registry; adding an app is configuration.
Dead code	Library calls broken by upgrades (some admin deletes always fail), old PHP migration tools, test routes and large blocks of hardcoded data still deployed.	Current libraries, strict TypeScript, lint and tests on every change; test and one-off tools are not rebuilt.
Data formats	Dates stored as text and compared alphabetically (some unpadded, which breaks comparisons); three payment-time formats; money as floating-point numbers; booleans as `0/1/"0"/"1"/""`; a date bug stamped payments with the year 2439.	Real dates in UTC, money in integer cents with a currency, real booleans and enums. Migration converts every known format and reports what it can't.
Data meaning	`is_trial` actually encodes the license state (trial / revoked / lifetime / yearly); "lifetime" is a date ~400 years away; one transaction field can hold five different kinds of value.	An explicit license model (plan, status, expiry; no expiry = lifetime), typed references, and a history of every activation.

Integrity	Missing indexes (every app request scans all playlists); a cleanup job deletes devices but leaves their playlists; the same MAC can be stored twice.	Indexes managed by migrations, correct cleanup, and a **unique MAC per app** enforced by the database. Past duplicates are merged during migration by fixed rules.
Settings	A loose key/value table with duplicate keys; payment credentials in plain text.	One typed config per app; secrets encrypted separately and never returned by any API.
Security	Credentials hardcoded in source; payment confirmations accepted without verifying the provider; maintenance routes reachable without login; missing ownership checks; unvalidated input; device tokens that never expire; unknown domains falling back to another app's database.	Secrets only in the environment / encrypted store; every payment verified with the provider and processed exactly once; explicit auth on every route; schema validation of all input; signed, expiring, app-bound tokens; exact domain-to-app mapping.
Reliability	Up to 24 processes each keep their own counters and caches; scheduled jobs run once per process (up to 24×); unhandled errors leave requests hanging or crash processes.	Shared state in Redis; jobs run exactly once in a separate worker; a global error handler always answers.
Abuse	Every unknown MAC creates a permanent device; defences delete all devices behind an IP and ban IPs permanently, while trusting spoofable IP headers.	Layered protection (section 4): edge rules, shared rate limits, self-expiring pending devices, quarantine instead of deletion.

3. Target architecture



- **API-only backend:** admin, reseller and customer UIs become separate frontends.
- **Central admins and resellers** with **one credit wallet usable in every app**, backed by an append-only ledger (every credit movement is recorded; no double-spending).
- **Legacy compatibility proven by tests:** every old endpoint is inventoried with its exact contract, and test vectors generated from the old code must be reproduced byte for byte.
- **Repeatable data migration** with verification reports; every migrated record keeps its old id.

4. Protection against DDoS and fake-device flooding

Edge	Server accepts traffic only via Cloudflare, so client IP/network info is trustworthy; rate-limit rules on registration; temporary IP blocks instead of permanent bans.
Shared rate limits	Per IP, network range, ASN, app and endpoint type — counted in Redis, shared by all servers.
Cheap rejection	Invalid payloads, malformed MACs and burst patterns rejected before touching the database.
Pending devices	Unknown MACs go to a collection that expires automatically. The app gets the same response as today; the device becomes permanent on its first real action (playlist added, activation, payment). Fake devices disappear by themselves — no deletion jobs.
Quarantine	Suspicious devices flagged for review instead of deleted; paid devices never removed automatically.
New app builds	Device attestation and signed, expiring tokens.

5. Technology stack

Technology	Why
Node.js 24 LTS	Same language as the existing code and the team; current long-term-support version, supported well beyond launch.
TypeScript (strict)	Most existing bugs are undefined values, wrong types and misspelled fields — the compiler catches these before the code runs.
NestJS	Provides the structure the old code lacks: modules, dependency injection (easy testing), guards for auth, interceptors for the legacy encodings, error filters so no request can hang; API and worker from one codebase.
Fastify (under NestJS)	~2–3× faster than Express — matters under attack; gives control over raw request bytes, needed to verify legacy request signatures.
MongoDB	All existing data is already in MongoDB, so migration is a transformation rather than a technology change; per-app databases keep apps isolated.
Redis	Fast shared memory for rate limits, caches and locks across all servers; never the source of truth; small and inexpensive.
BullMQ	Background jobs and schedules on Redis: each job runs exactly once, survives restarts, retries on failure, and runs outside the API.
zod	Validates configuration at startup (missing secrets stop the app) and all incoming data (closes injection bugs).
pino	Fast structured logging with request ids; secrets and personal data redacted.
Docker	Identical environments for every developer and server; the backend ships as an image.
Automated tests + CI	Unit, integration and compatibility tests run on every change; nothing merges unless they pass.

6. Roadmap

#	Milestone	Outcome
1	Discovery	Inventory of every endpoint and its exact contract; legacy encodings specified with test vectors; sample data profiled; field mappings
2	Foundation	Production-grade empty service: config, logging, error handling, MongoDB, Redis, worker, tests, Docker, CI

3	Tenancy	Domain → app resolution, central registry, encrypted per-app secrets, strict isolation
4	Core domain	Devices, licensing, playlists, content, translations, app updates
5	Accounts & wallet	Central admins and resellers, single credit wallet, codes, coupons, customer accounts
6	Legacy compatibility	Every legacy app route, byte-compatible, proven by tests
7–8	Store, partners, payments	Store contract preserved; transfers as internal operations; all payment providers behind one interface
9–11	Abuse, admin APIs, jobs	Section 4; APIs for the new frontends; all background work single-run and observable
12–13	Migration & hardening	Repeatable data migration with verification; load tests, failure drills, security review
—	Phase 2	Fix all tagged bugs and security gaps; remove dead endpoints — required before go-live
14	Cutover	App-by-app switch from a rehearsed runbook, with rollback